High-Level Design (HLD)

1. **System Overview:**
   Problem Statement
   Customers often struggle to get accurate answers from large documents (PDFs, manuals, FAQs).
   Traditional chatbots fail because they don't use document context.

   Solution
   A **RAG-based system** that:

- Reads PDF knowledge base
- Retrieves relevant content
- Generates contextual answers
- Uses **LangGraph for workflow**
- Supports **Human-in-the-Loop (HITL**

2. **Architecture Diagram**

```
User Query
   ↓
User Interface (CLI)
   ↓
LangGraph Workflow Engine
   ↓
[Process Node]
   ↓
Retriever (ChromaDB)
   ↓
Relevant Chunks
   ↓
LLM (Groq - LLaMA 3.1)
   ↓
Answer Generated
   ↓
[Output Node]
   ↓
User

      ↘ (if low confidence)
       Human-in-the-Loop (HITL)
```

3. Component Description

1. Document Loader

- Loads PDF using PyPDFLoader
- Converts into raw text documents

2. Chunking Module

- Splits text into smaller chunks
- Uses:
  - chunk size: 500
  - overlap: 50

3. Embedding Model

- Converts text into vectors
- Model:
  - all-MiniLM-L6-v2

4. Vector Database (ChromaDB)

- Stores embeddings
- Enables fast similarity search

5. Retriever

- Fetches top-k relevant chunks
- Based on user query

6. LLM (Groq)

- Model: llama-3.1-8b-instant
- Generates answers using retrieved context

7. LangGraph Workflow Engine

- Controls execution flow
- Nodes:
  - Process Node
  - Output Node

8. Routing Layer

- Decides:
  - Normal response
  - Escalation to human

9. HITL (Human-in-the-Loop)

- Triggered when:
  - Low confidence

    - No relevant data
- Human provides final answer

4. **Data Flow**

   PDF Processing Flow:

   PDF → Loader → Chunking → Embeddings → ChromaDB

   Query Flow:

   User Query
   ↓
   Retriever → Relevant Chunks
   ↓
   LLM → Answer
   ↓
   LangGraph → Output / HITL

5. **Technology Choices**

Component	Technology	Reason
Loader	PyPDF	Easy PDF parsing
Chunking	LangChain Splitter	Efficient text splitting
Embeddings	HuggingFace	Free + fast
Vector DB	ChromaDB	Lightweight & local
LLM	Groq (LLaMA 3.1)	Fast inference
Workflow	LangGraph	Control logic
Env	Python + VS Code	Simple dev setup

6. Scalability Considerations

Large Documents

- Use smaller chunk sizes
- Batch embedding

High Query Load

- Cache responses
- Use faster LLM

Latency

- Reduce chunk size
- Limit top-k retrieval